

MPI Communication and Memory Design Assignment 1

COMP3030 – Paralleled and Distributed Computing

Nicholas Koulianos
A1875019

Table of Contents:

1. Overview	1
2. Part 1A.....	1
3. Part 1B.....	1
4. Design Comparison	2
5. Validation	2
6. Appendices	3
6.1. Appendix A	3
6.2. Appendix B	3
6.3. Appendix C	3

Table of Tables:

Table 1: Testing results of all MPI Methods	3
---	---

Table of Figures:

Figure 1: Ring-implementation of Part1A diagram	3
Figure 2: Ring-buffer implementation of Part1B diagram	3

1. Overview

The n-body problem is a benchmark test for computing hardware and software, lending particularly well to parallelization. The initial code for this report utilizes MPI to evenly distribute the total particles through data parallelism onto separate processes, wherein each process can solve a portion of the total calculations at the same time. This reduces the total calculation time fractionally while a synchronized communication updates every process of the global problem data. This report will be identifying how a ring transfer implementation can serve to simplify this problem further, making use of independent memory storage on each process.

2. Part 1A

In the original MPI implementation provided, the unionizing of local data on each process is complete using the MPI Allgather method. For each time step, once the forces on local particles are calculated and their velocities updated, MPI Allgather is called to pull all local position data from each process, and immediately scatter it to every process; updating every process's global position matrix.

In the ring implementation, MPI Allgather is to be replaced with a point-to-point ring communication, in which position data is passed through neighbour interactions in one direction. Beginning with the master process (rank 0), the first set of updated local positions are sent to its successive neighbour (the process with rank 1). The master process then awaits to receive a set of local positions from the highest rank core, its preceding neighbour. Meanwhile, the rank 1 process receives process 0's local positions and directly places them into its local copy of the global position matrix, placed at an indexed position dictated by rank. It then in turn sends its updated local positions to the neighbouring process and this repeats until a full cycle is complete. A number of cycles are performed equal to the number of processes - 1, in which each process transfers the local positions it received in the last cycle and receives correspondingly from its neighbour. By the end of all cycles, each process will have received all sets of local positions until each process independently stores an identical global position matrix. This then informs the force calculations and particles updates for the next timestep, in which the ring model repeats.

The variables "next", "previous", "next_block" and "previous_block" are the deciding arithmetic that ensure the ring transfers operate correctly. "Next" and "Previous" are unchanging variables calculated unique to each process based on their rank. "Next" is defined simply as current rank + 1, taking the modulus by comm_size to ensure this value wraps around the ring if the current process has the highest rank. For "Previous", this is calculated similarly by taking the current rank - 1, but padded with comm_size prior to the modulus to protect against negative rank values. "next_block" and "previous_block" are similarly defined, but on a per transfer basis with a transfer index subtracted from both variables. These variables, on a per process basis, track which block of local positions each process is receiving and which block they are sending based on the number of transfers complete. For instance, each process begins by sending its own local position array, and then for all subsequent transfers, sends the block it most recently received until eventually receiving local positions from its succeeding neighbour having circulated the ring. The importance of these variables changing with each transfer is they identify where in each process's local copy of global positions a receiving block should be inserted, utilizing the code `pos + next_block/receiving_block * loc_n`. This takes the address of the first element of the global position array and uses the next_block/receiving_block variables multiplied by the size of a local position array to find the first element of where the data should be updated.

Finally, the ring implementation handles sending and receiving data separately depending on whether it's the master process or not. If every process in the ring implementation waited to receive first prior to sending, the ring would never initiate and effectively deadlock. This occurs as the master process awaits to receive data that its prior neighbour can never produce, as it in turn awaits for a data transfer that needs to begin with the master process. To amend this issue, the master process has the exception that it first sends data, and then waits to receive. This initiates the ring transfer, and while the master process waits to receive, all other processes can continue transferring until eventually completing the cycle. A diagram of the ring implementation can be seen in Appendix A.

3. Part 1B

In the reduced-memory implementation, each process can no longer fetch and store data in a "global" position array. The major caveat of this current memory allocation model is its effectively using distributed-memory and data-parallelism to emulate shared-memory, but inefficiently so. While each process separately calculates force from its own portion of positions, the processes don't share a single global position array and instead all contain a local copy of it. Therefore, for each time step, they must circulate their portion of the data around the ring to rebuild the global position array on every process.

To improve the efficiency of this process, the reduced-memory design creates global arrays for position, masses and velocities once: when calling the Get_init_cond() method to generate

the initial problem, which is then distributed in unique portions to each process with the global arrays then erased. Each process only permanently stores a unique array of positions, masses, forces and velocities specific to the data it was distributed.

For handling the n-body arithmetic, the original model in Part 1A would compute forces and update particles once at the beginning of each time step using global arrays, prior to circulating its new position data around the ring. However, in a reduced-memory implementation, forces cannot be computed once per timestep as each process will never have access to all global position and mass data at once. Instead, forces are calculated per ring transfer based on the received positions and masses of the neighbouring process and accumulated overtime. To facilitate this behaviour, each process now requires two temporary memory allocations that store entering and exiting data circulating around the ring. In the provided code implementation, these arrays are suffixed by 'send' and 'recv'. These are temporary, rather than local, as their data changes on every send and receive between processes and are stored only briefly to compute forces, before being sent onward to the next neighbour. In the coded implementation, this is handled by a third temporary array to swap the send and receive arrays at the end of each force computation. This makes it so the recently received masses and positions are sent forward on the next transfer. This can be seen in Appendix B in the form of a diagram.

In the original ring implementation, only positions were required to be transferred between processes as they were the only changing global data required for computing forces and updating particles. However, in a reduced-memory model, this global array of masses stored on each individual process is now replaced by a local array of masses (unique to the distributed data given to each process). To still facilitate global access, a given process must receive local masses and local positions from its neighbour's particles, calculate the forces incurred by them on its own particles, and then pass the local masses and positions onto its neighbour. This transfer process repeats until each process has updated its forces based on the masses and positions of all particles across all processes, and then finally updates its local particles before the next time step. Furthermore, outputs are now handled similar to initialization, in which a global matrix is created temporarily, filled with local positions and velocities from every process, and then outputted before freeing the global arrays.

4. Design Comparison

The per-process memory requirements between the original MPI solver and ring implementation, despite having different transfer methods, are effectively identical. Both implementations still maintain global mass and position arrays on every process, in addition to their unique local arrays. Furthermore, in the original MPI implementation, every process calls MPI_allgather, meaning in a single time step, every process will receive $\text{comm_size} - 1$ times and transfers $\text{comm_size} - 1$ times. Likewise, in the ring implementation, each process performs one send and one receive per ring transfer, which occurs $\text{comm_size} - 1$ times.

However, in the reduced-memory implementation, global arrays are disbanded entirely after initialization. This means each process now contains arrays for mass and position of only size $n/\text{comm_size}$, as opposed to size $n + n/\text{comm_size}$ for global and local arrays respectively. However, to facilitate this new design, each process now allocates five temporary arrays to temporarily hold data of size $n/\text{comm_size}$: two for position, two for mass, and one for swapping data. Furthermore, without a global mass array stored on each process, more data is now required to be transferred between processes (albeit of type double, less memory than a vector type).

While this reduced-memory model may prove inefficient for problems where the comm_size is small (close in value to the number of added temporary arrays), the benefits of this design grow exponentially with increasing comm_size as the number of additional arrays remains constant. Additionally, increasing comm_size creates smaller partitions of the same n particles, making the tradeoff of additional data transfer less severe. In Table 1, it can be seen on average that across various testing condition, the design implementation created in Part 1A provides the quickest return on results, with Part 1B's design often being the slowest. This is expected as Part 1B's memory reduction relies on approximately 50% more data being transferred. This could likely be improved by only transferring masses on the first timestep (given they are static values) as opposed to every timestep, or creating a vector to move both mass and positions in one transfer, although the latter option will not impact memory usage.

5. Validation

To validate the results from both new implementations of the MPI method, the baseline code was provided three unique sets of initial data and the results produced were considered datum for comparison. Results were considered valid and consistent as long as they matched the default MPI method within 4 significant figures, as per typical engineering tolerances. These datasets were chosen with specific edge cases in mind, testing different particle quantities, mass ranges and initial distances. The initial conditions were then tested under a range of process quantities and timestep quantities and sizes, providing 9 total datums for comparison. These were selected with whitebox testing in mind, such as for the 1 process case. The final results of these numerical comparisons can be seen below in Table 1.

6. Appendices

6.1. Appendix A

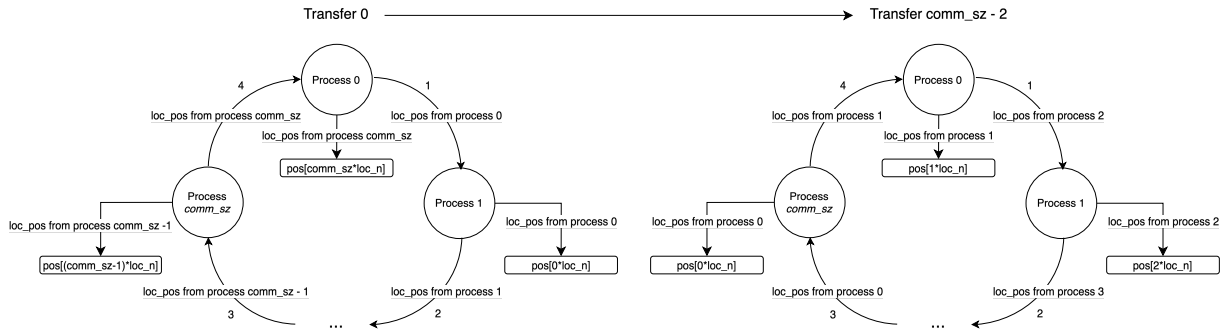


Figure 1: Ring-implementation of Part1A diagram

The above diagram displays how local positions are transferred through the ring implementation in Part1A, used to update global position arrays, and then transferred forward on future transfer iterations. Arrows dictating movement of data have been labelled in order of occurrence, showing the master process to first initiate the data transfer.

6.2. Appendix B

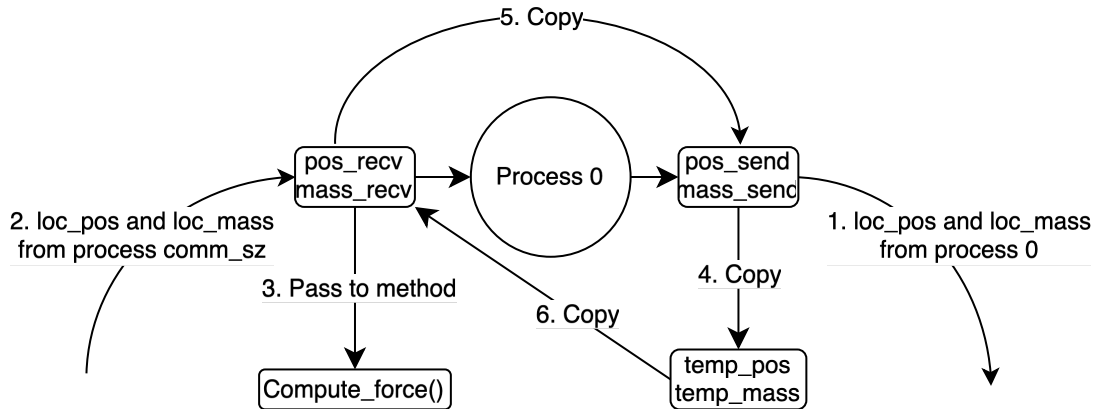


Figure 2: Ring-buffer implementation of Part1B diagram

The above diagram displays how local positions and masses are continuously cycled around the ring without global arrays in Part1B. The diagram is localized to one process, where as the actual code implementation would repeat the illustrated process on every core, akin to the structure seen in Appendix A. The movement of data has been numbered in order of chronological occurrence. For the example shown in the diagram, the sources of data are accurate to the first transfer cycle, transfer 0.

6.3. Appendix C

Table 1: Testing results of all MPI Methods

Particle Count	Parameters	Program	Elapsed Time (Seconds)
4	1 Process	Basic	1.01e-4
	10 steps	Part1a	1.38e-4
	0.01 size	Part1b	1.37e-4
	2 Process	Basic	8.09e-4

96	10 steps	Part1a	4.73e-4
	0.1 size	Part1b	6.32e-4
	4 Process	Basic	1.91e-4
	5 steps	Part1a	1.11e-4
	0.01 size	Part1b	2.64e-4
	2 Process	Basic	8.37e-4
	10 steps	Part1a	8.94e-4
	0.01 size	Part1b	4.46e-3
	4 Process	Basic	1.43e-3
	20 steps	Part1a	1.30e-3
	0.1 size	Part1b	1.13e-3
	8 Process	Basic	1.13e-2
	10 steps	Part1a	1.06e-3
	0.01 size	Part1b	7.54e-3
	1 Process	Basic	1.82e-4
	10 steps	Part1a	2.03e-4
	0.01 size	Part1b	2.58e-4
	4 Process	Basic	5.27e-4
20	20 steps	Part1a	3.66e-4
	0.1 size	Part1b	7.40e-4
	5 Process	Basic	7.05e-4
	10 steps	Part1a	4.81e-4
	0.01 size	Part1b	6.47e-4

The above table provides the final elapsed times of all performed tests across all 3 of the major programs. For a given test under given parameters, the fastest running program's time has been highlighted in green.